

1.1 Ejercicio 1.1. Programación estructurada y orientada a objetos

Dos diferencias fundamentales entre ambos paradigmas:

1. **Organización del código.** La programación estructurada organiza el programa como una secuencia de procedimientos o funciones que operan sobre datos que se les pasan explícitamente. La programación orientada a objetos agrupa datos y comportamiento en una misma unidad (la clase), de modo que cada objeto es responsable de su propio estado.
2. **Mecanismos de reutilización.** La programación estructurada no ofrece un mecanismo nativo de reutilización más allá de la llamada a funciones; la orientación a objetos incorpora herencia y polimorfismo, que permiten extender y especializar comportamiento sin duplicar código.

La orientación a objetos resulta más adecuada para programas de gran tamaño porque el encapsulamiento reduce el acoplamiento entre partes del sistema: los cambios internos de una clase no deberían afectar al resto del programa mientras se mantenga su interfaz pública. Esto facilita el mantenimiento, la división del trabajo entre varios desarrolladores y la evolución del software a largo plazo.

1.2 Ejercicio 1.2. La máquina virtual de Java

La JVM es el componente que ejecuta el bytecode generado por el compilador de Java (archivos `.class`). Actúa como una capa de abstracción entre ese bytecode y el sistema operativo o hardware concreto, encargándose de traducirlo a instrucciones nativas (mediante interpretación y/o compilación *just-in-time*), de gestionar la memoria (recolector de basura) y de verificar la validez del bytecode antes de ejecutarlo.

La diferencia con lenguajes que compilan directamente a código nativo (como C) es que, en esos lenguajes, el compilador genera ya el código máquina específico para una arquitectura y sistema operativo concretos, ejecutable directamente por el procesador. En Java, el compilador (`javac`) no genera código nativo, sino bytecode independiente de la plataforma; es la JVM instalada en cada sistema la que se encarga de convertirlo en instrucciones ejecutables por ese sistema en concreto. Esto añade una capa intermedia, pero a cambio proporciona portabilidad.

1.3 Ejercicio 1.3. Estructura mínima de un programa Java

Un programa Java necesita, como mínimo:

- **Una declaración de clase**, que actúa como contenedor del código.
- **Un método `main` con la firma exacta** `public static void main(String[] args)`, que es el punto de entrada que invoca la JVM al arrancar el programa.
- **Coincidencia entre el nombre del archivo fuente y el de la clase pública** que contiene, si dicha clase se declara `public`.

El método `main` debe ser `static` porque la JVM lo invoca sin haber creado antes ningún objeto de la clase; `void` porque no devuelve ningún valor al sistema operativo; y el parámetro `String[] args` permite recibir argumentos desde la línea de comandos.

1.4 Ejercicio 1.4. Análisis de un programa Java sencillo

Considera el siguiente programa:

```
public class Prueba {
    public static void main(String[] args) {
        System.out.println("Java");
    }
}
```

- a) El programa contiene **una única clase**, Prueba.
- b) El punto de entrada es el método `main(String[] args)`.
- c) La instrucción que produce la salida por pantalla es `System.out.println("Java");`.

1.5 Ejercicio 1.5. Compilación y ejecución de programas Java

- a) Para compilar el fichero `Ejemplo.java`:

```
javac Ejemplo.java
```

- b) Para ejecutar el programa ya compilado:

```
java Ejemplo
```

En el primer paso, `javac` analiza el código fuente y, si no contiene errores, genera un fichero `Ejemplo.class` con el bytecode correspondiente. En el segundo paso, el comando `java` arranca una instancia de la JVM, carga la clase `Ejemplo` y comienza la ejecución invocando su método `main`.

Nota. Nótese que a `java` se le indica el nombre de la clase (sin extensión `.class`), no el del fichero.

1.6 Ejercicio 1.6. JDK y JRE

El **JRE** (*Java Runtime Environment*) contiene lo necesario para *ejecutar* programas Java ya compilados: la JVM y las bibliotecas estándar. El **JDK** (*Java Development Kit*) incluye, además de todo lo anterior, las herramientas necesarias para *desarrollar* software, en particular el compilador `javac`, además de utilidades como `javadoc` o un depurador.

No basta con instalar el JRE para desarrollar en Java porque el JRE no incluye `javac`: sin compilador no es posible traducir el código fuente a bytecode, y por tanto no se puede generar el fichero `.class` que la JVM necesita para ejecutar el programa.

1.7 Ejercicio 1.7. Portabilidad del bytecode

El fichero `.class` generado en Windows puede ejecutarse sin modificaciones en Linux o macOS porque no contiene código máquina específico de una arquitectura, sino **bytecode independiente de la plataforma**. El componente que hace posible esta portabilidad es la **JVM**: cada sistema operativo dispone de su propia implementación de la máquina virtual, capaz de interpretar ese mismo bytecode y traducirlo a las instrucciones nativas de su plataforma. De ahí el lema “*compílolo una vez, ejecútalo en cualquier sitio*”.

1.8 Ejercicio 1.8. Tipos de errores

- a) **Error lógico.** El programa compila y se ejecuta sin interrupciones, pero el resultado es incorrecto para cualquier entrada; la causa es un fallo en el algoritmo (por ejemplo, una inicialización o una condición de bucle mal planteadas), no algo que el compilador o la JVM puedan detectar.
- b) **Error de compilación.** `int x = "hola";` intenta asignar un valor de tipo `String` a una variable de tipo `int`. Java es un lenguaje de tipado estático, por lo que esta incompatibilidad se detecta durante la compilación y `javac` rechaza generar el bytecode.
- c) **Error de ejecución.** Acceder al índice 10 de un array de 5 elementos es sintácticamente correcto y no puede detectarse en tiempo de compilación, ya que el tamaño del array o el índice pueden depender de datos conocidos solo en tiempo de ejecución. La JVM lanza una excepción (`ArrayIndexOutOfBoundsException`) al llegar a esa instrucción.